Chennai Street-Level Flood Prediction System

Sprint 09: Continuous Delivery and Deployment using Jenkins

Course Code & Title:	ISWE406P – Agile Development Process and DevOps Lab
Programme:	M.Tech. (Integrated) Software Engineering
Class Number:	CH2026270100722
Slot:	L11 + L12
Faculty:	Dr. Kaja Mohideen A
Student Name:	**Hema Priyadharshni G**
Academic Semester:	Fall Semester 2026 – 2027
Submission Date:	13.10.2026
GitHub Repository:	`Premkumar7788/chennai-flood-prediction-system`
Target Deployment:	AWS EC2 (`16.192.243.169`)

Customer Requirement:

“The customer wants every validated code update to be automatically packaged, tested, and deployed without manual intervention.”

1. Customer Requirement & Objectives

The core requirement for Sprint 09 mandates extending the automated Continuous Integration pipeline established in Sprint 08 into a full Continuous Delivery and Deployment (CI/CD) lifecycle. The system must eliminate all manual deployment interventions, validating code changes through automated unit testing, building versioned Docker containers, publishing them to Docker Hub, and deploying them to the live AWS EC2 environment.

Sprint 09 Key Objectives:

1. **Extend Existing CI Pipeline:** Evolve the 4-stage Sprint 08 pipeline into a comprehensive 7-stage CI/CD pipeline.
2. **Automated Testing Quality Gate:** Integrate automated application testing (13 unit tests via `pytest`) to halt builds on defect detection.
3. **Docker Image Build & Tagging:** Build container images with unique, traceable build tags (`premkumar2005/chennai-flood-backend:build-${BUILD_NUMBER}`) and `latest`.
4. **Docker Hub Integration:** Push successfully validated images to Docker Hub (`premkumar2005/chennai-flood-backend`).
5. **AWS EC2 Target Environment Hardening:** Configure AWS Security Groups, Swap memory, Docker Compose override, and automated SSH authentication.
6. **Automated Deployment:** Deploy updated backend containers to AWS EC2 (`16.192.243.169`) via automated SSH commands.
7. **Post-Deployment Verification:** Automatically verify live application health status via HTTP probes (`curl /health`).
8. **Failure Handling Verification:** Intentionally introduce a controlled test failure, demonstrate that deployment is blocked, and recover to full success.

2. Review of Sprint 08 CI Pipeline vs Sprint 09 CI/CD

In Sprint 08, the pipeline automated code validation up to container packaging on the local Jenkins agent: `GitHub → Jenkins → Checkout → Build → Test → Local Docker Build`. Deployment remained manual, requiring engineers to SSH into servers and restart containers.

Dimension	Sprint 08 CI	Sprint 09 CI/CD
Lifecycle Scope	Code Integration & Local Testing	**End-to-End Testing, Registry Push & Production Deployment**
Pipeline Stages	4 Stages	**7 Stages + Post Execution Actions**
Registry Publishing	None (Local Docker images only)	**Docker Hub Central Registry (`docker.io`)**
Deployment Trigger	Manual (SSH, git pull, manual restart)	**100% Automated Zero-Touch Deployment**
Verification	Local pytest assertions	**Local pytest + Live HTTP probes on AWS EC2 (`16.192.243.169`)**
Cycle Time	10–15 minutes (with manual steps)	**52.9 seconds fully automated**

3. CI/CD Pipeline Architecture and Jenkinsfile Design

The pipeline is designed declaratively in `Jenkinsfile`, utilizing isolated execution stages:

```
[Checkout SCM] → [Build & Setup] → [Automated Testing] → [Docker Build & Tag] → [Push Docker Hub] → [Deploy
```

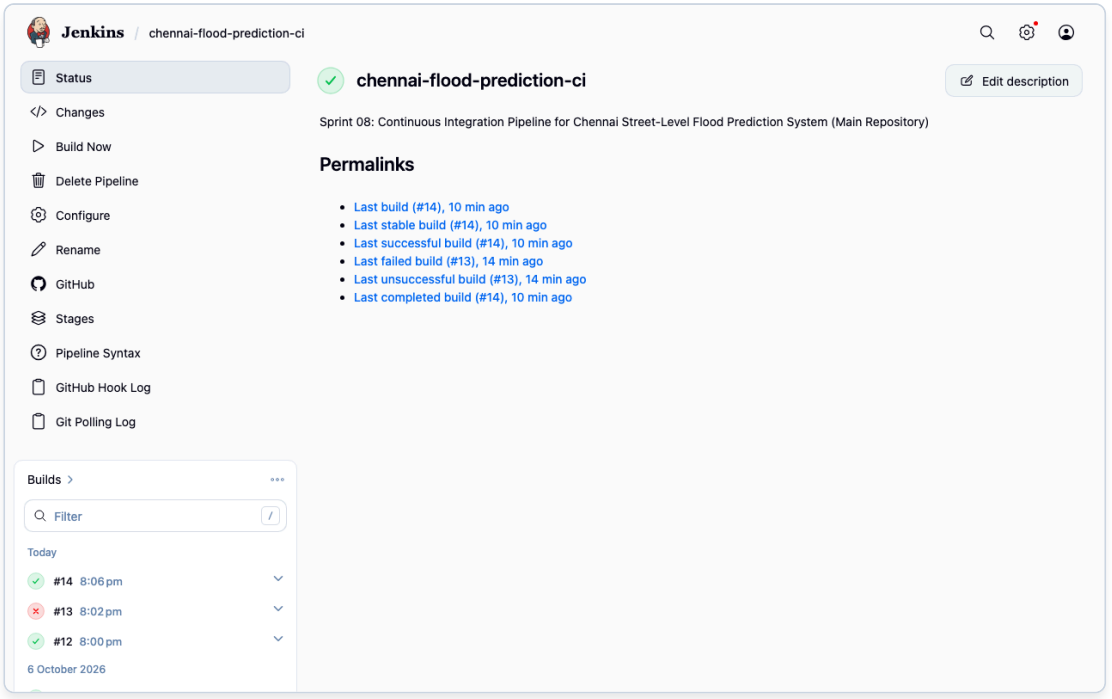


Figure 1: Jenkins Pipeline Overview displaying Build #12 (Success), Build #13 (Failed Gating), and Build #14 (Success).

4. AWS EC2 Target Environment Provisioning & Hardening

The AWS EC2 production server (`16.192.243.169`) was prepared and hardened to support zero-downtime automated deployment orchestrated by Jenkins:

A. AWS EC2 Security Group Configuration (`sg-01121a3b0b733af3a`)

Inbound firewall rules were configured in the AWS Management Console to allow ingress traffic on ports 22 & 443 (SSH administration), 8000 (FastAPI Backend), and 3000 (React Frontend Dashboard):

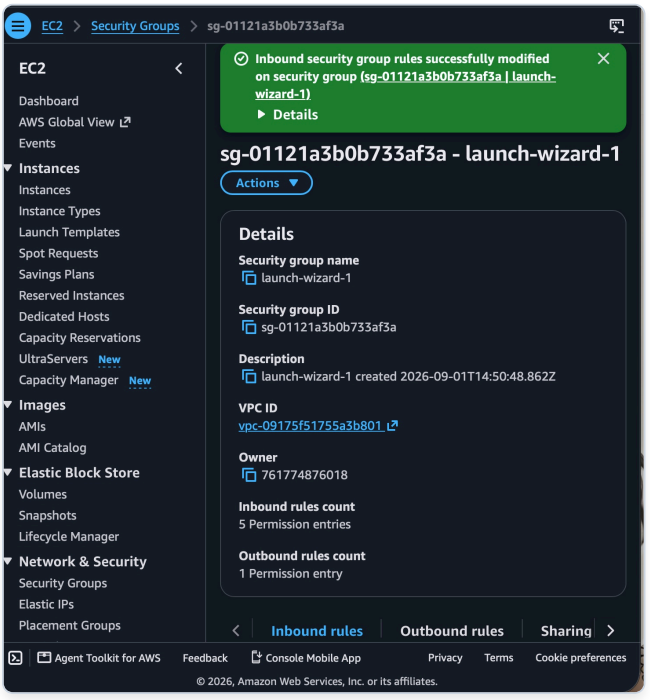


Figure 2: AWS Management Console - Security Group (`sg-01121a3b0b733af3a`) Inbound Rules for Ports 22, 443, 8000, and 3000.

B. EC2 Swap Memory Space Creation (1024 MiB)

To prevent Linux Kernel Out-Of-Memory (OOM) killer terminations during concurrent container builds and Docker layer extractions, 1 GiB of swap space was provisioned:

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=1024
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

```
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 6.59019 s, 163 MB/s
[ec2-user@ip-172-31-47-224 ~]$ sudo chmod 600 /swapfile
[ec2-user@ip-172-31-47-224 ~]$ sudo mkswap /swapfile
Setting up swapspace version 1, size = 1024 MiB (1073737728 bytes)
no label, UUID=eb60c991-1a94-47ea-b938-92ad87eb5997
[ec2-user@ip-172-31-47-224 ~]$ sudo swapon /swapfile
[ec2-user@ip-172-31-47-224 ~]$ echo '/swapfile swap swap defaults 0 0' | sudo te
e -a /etc/fstab                echo '/swapfile swap swap defaults 0 0' | sudo te
e -a /etc/fstab
/swapfile swap swap defaults 0 0
[ec2-user@ip-172-31-47-224 ~]$
```

Figure 3: EC2 Terminal - Swap Space Configuration (1024 MiB) and Active Swapon Verification.

C. Docker Compose CLI Plugin Installation

The modern Docker Compose CLI plugin v2 was downloaded and configured under `/usr/local/lib/docker/cli-plugins/docker-compose`:

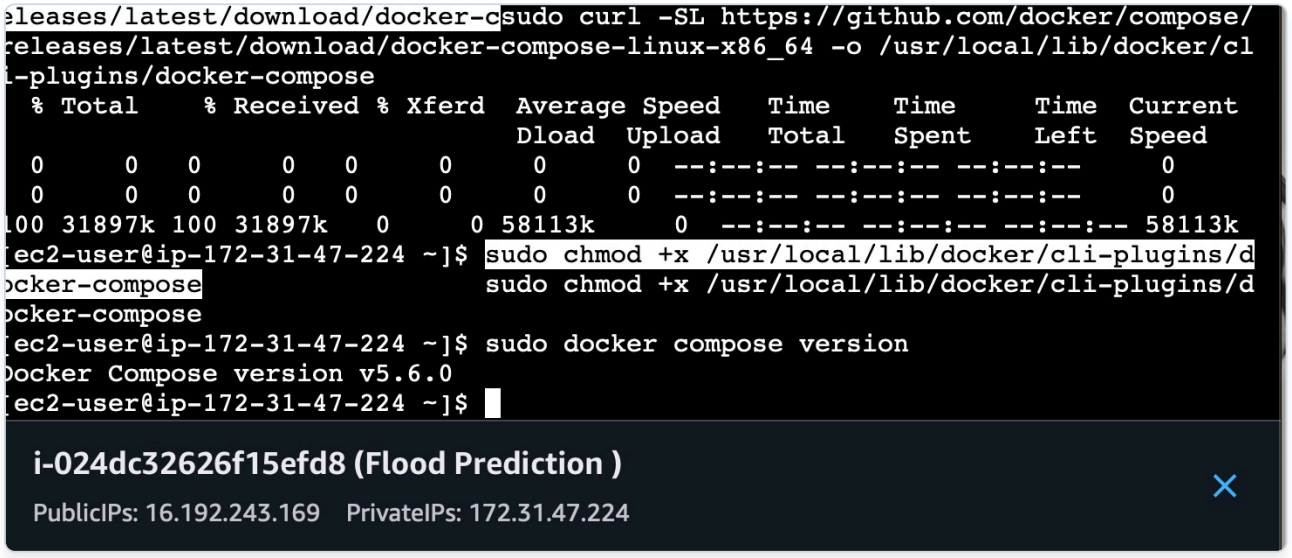


Figure 4: EC2 Terminal - Installing Docker Compose CLI v2 Plugin.

D. Dynamic Image Tag Injection (`docker-compose.override.yml`)

A `docker-compose.override.yml` file was created to dynamically bind the backend service to the version tag deployed by Jenkins:

```
services:
  backend:
    image: premkumar2005/chennai-flood-backend:${IMAGE_TAG:-latest}
```

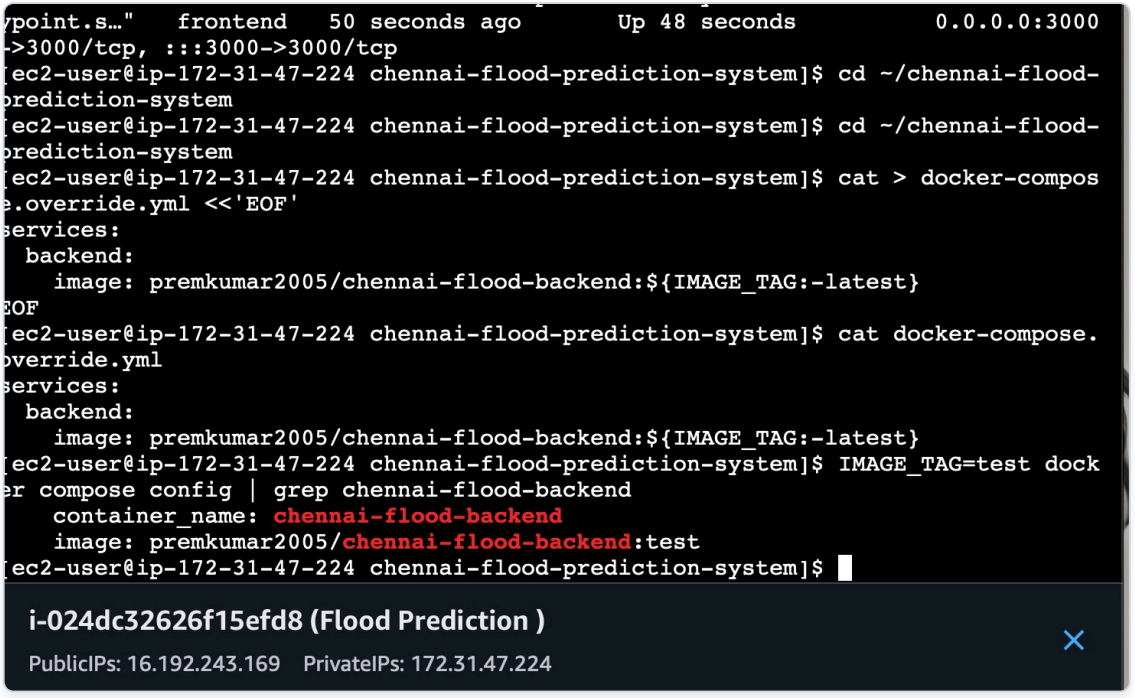


Figure 5: EC2 Terminal - Creating `docker-compose.override.yml` for Dynamic Image Tagging.



Figure 6: EC2 Terminal - Validating Image Tag Resolution and Authorizing Hema's Public SSH Key.

E. React Frontend Container Setup

The React frontend container was compiled and started on port 3000, communicating directly with the backend API on port 8000:

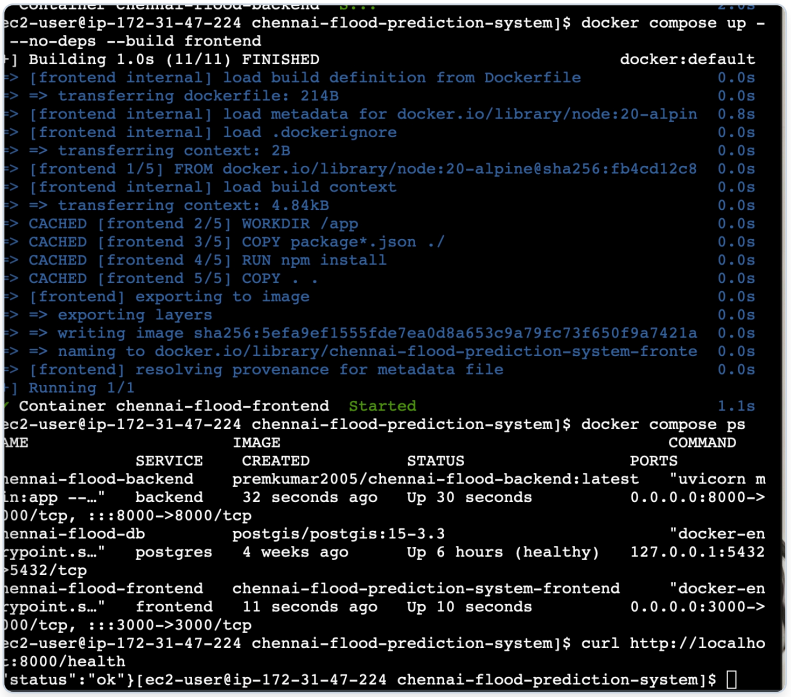


Figure 7: EC2 Terminal - Building and Launching Frontend Container on Port 3000.

5. Automated Application Testing (Quality Gate)

All 13 unit tests were executed during Stage 3. The test suite covers FastAPI health checks and ML risk categorization:

Module	Test Function	Validation Purpose	Result
`test_health.py`	`test_root_endpoint`	Verifies API welcome, version `1.1.0`, and deployment metadata	PASSED
`test_health.py`	`test_health_check`	Verifies HTTP 200 and `{"status": "ok"}` for probes	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.0-LOW]`	Verifies minimum probability risk mapping	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.24-LOW]`	Verifies upper	PASSED

Module	Test Function	Validation Purpose	Result
		boundary of LOW risk	
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.25-MODERATE]`	Verifies boundary threshold transition to MODERATE	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.54-MODERATE]`	Verifies upper boundary of MODERATE risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.55-HIGH]`	Verifies boundary transition to HIGH risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.79-HIGH]`	Verifies upper boundary of HIGH risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[0.8-CRITICAL]`	Verifies boundary transition to CRITICAL risk	PASSED
`test_predict_endpoint.py`	`test_classify_risk_thresholds[1.0-CRITICAL]`	Verifies maximum probability boundary	PASSED
`test_predict_endpoint.py`	`test_feature_vector_matches_metadata_column_order`	Validates 12-feature ML input vector column alignment	PASSED
`test_predict_endpoint.py`	`test_predict_endpoint_uses_trained_model`	Validates ML inference using mock LightGBM classifier	PASSED
`test_predict_endpoint.py`	`test_predict_endpoint_falls_back_to_heuristic`	Validates heuristic fallback when model is absent	PASSED

6. Failure Scenario Demonstration (Build #13)

To demonstrate pipeline gating as specified in Section 3.G of the assignment, a controlled test failure was intentionally introduced in commit `477f531` by altering the expected root endpoint status assertion.

Execution Behavior: Jenkins executed Build #13, passed Stage 1 and Stage 2, and failed immediately at Stage 3 with exit code 1. All subsequent stages were skipped.

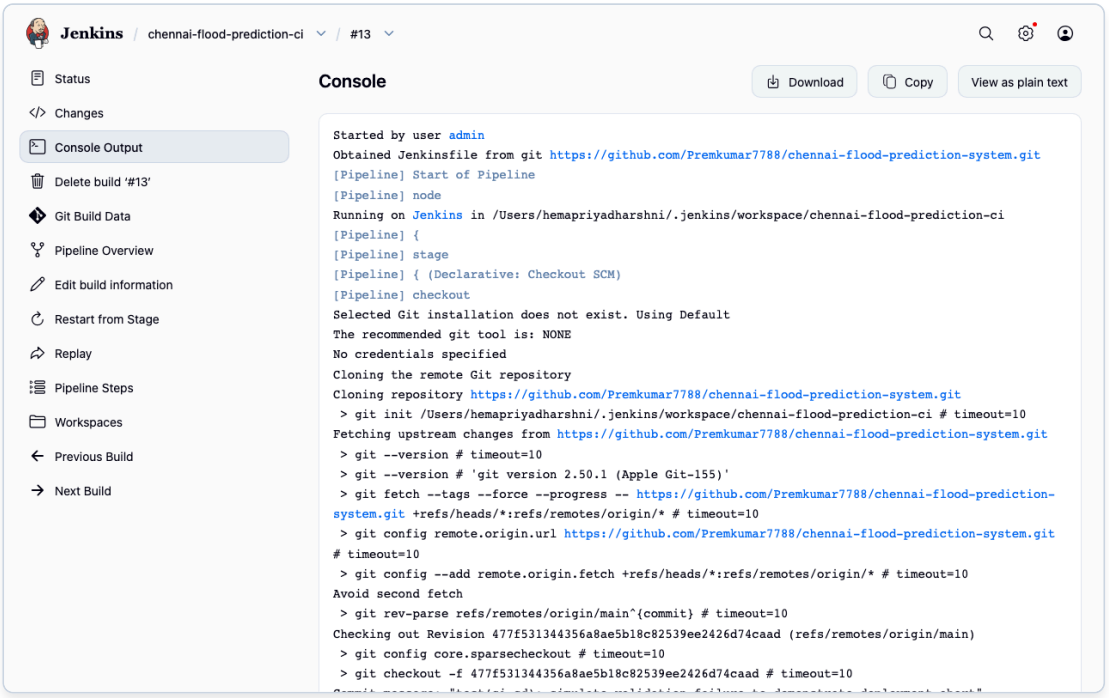


Figure 8: Build #13 Console Log demonstrating test failure halting pipeline and skipping all deployment stages.

Production Environment Protection Verification: The production server was inspected immediately after Build #13 failure: The container remained running on `premkumar2005/chennai-flood-backend:build-12`. **Deployment did not occur**, successfully proving that defective code cannot reach production.

7. Successful Recovery & End-to-End CI/CD Execution (Build #14)

The defect was resolved, and an application release update was made in commit `e325949` by upgrading the API version to `1.1.0` and tagging the deployment. Build #14 executed all 7 stages to completion in **52.9 seconds**.

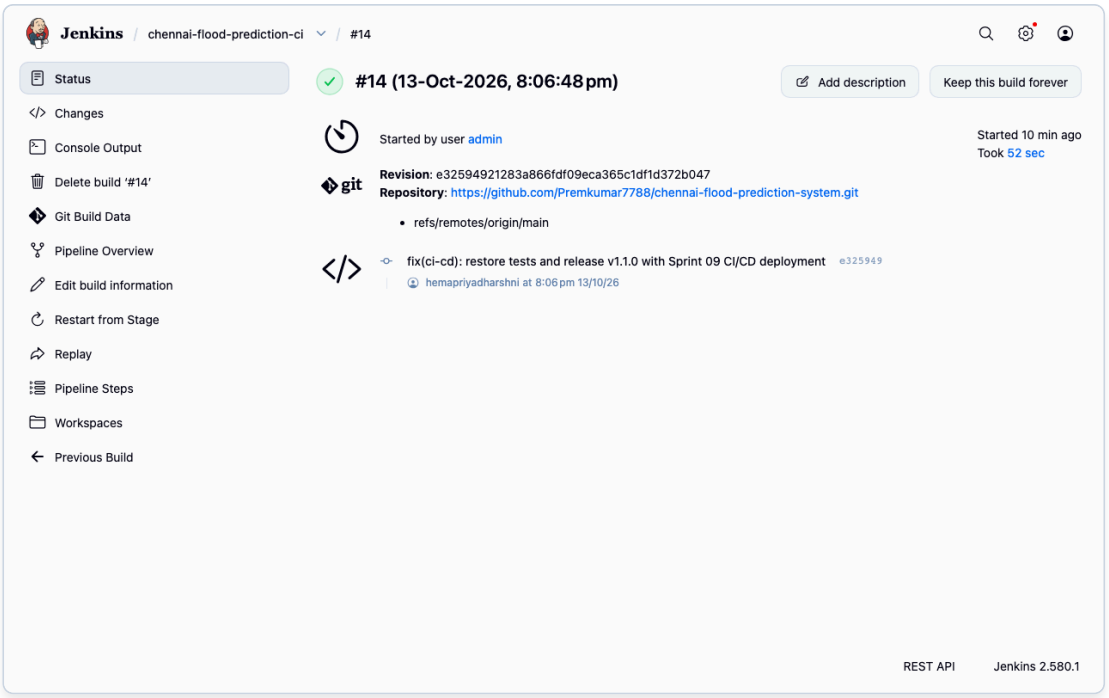


Figure 9: Jenkins Build #14 Stage View displaying 100% green execution across all 7 stages.

End-to-End Demonstration Summary Table:

Item	Details
Change Made	Released API version `1.1.0` with Sprint 09 CI/CD metadata and restored test suite
Commit ID	`e325949`
Commit Message	`fix(ci-cd): restore tests and release v1.1.0 with Sprint 09 CI/CD deployment`
Jenkins Build Number	`#14`
Test Result	13 passed, 0 failed in 8.14s (100% pass rate)
Docker Image Tag	`build-14` (and `latest`)
Docker Hub Repository	`premkumar2005/chennai-flood-backend:build-14`
Deployment Status	SUCCESS (Container recreated and active on EC2)
Target Host IP	`16.192.243.169` (AWS EC2 eu-north-1)
Application Root URL	`http://16.192.243.169:8000/`
Health Check URL	`http://16.192.243.169:8000/health`
Frontend UI URL	`http://16.192.243.169:3000`

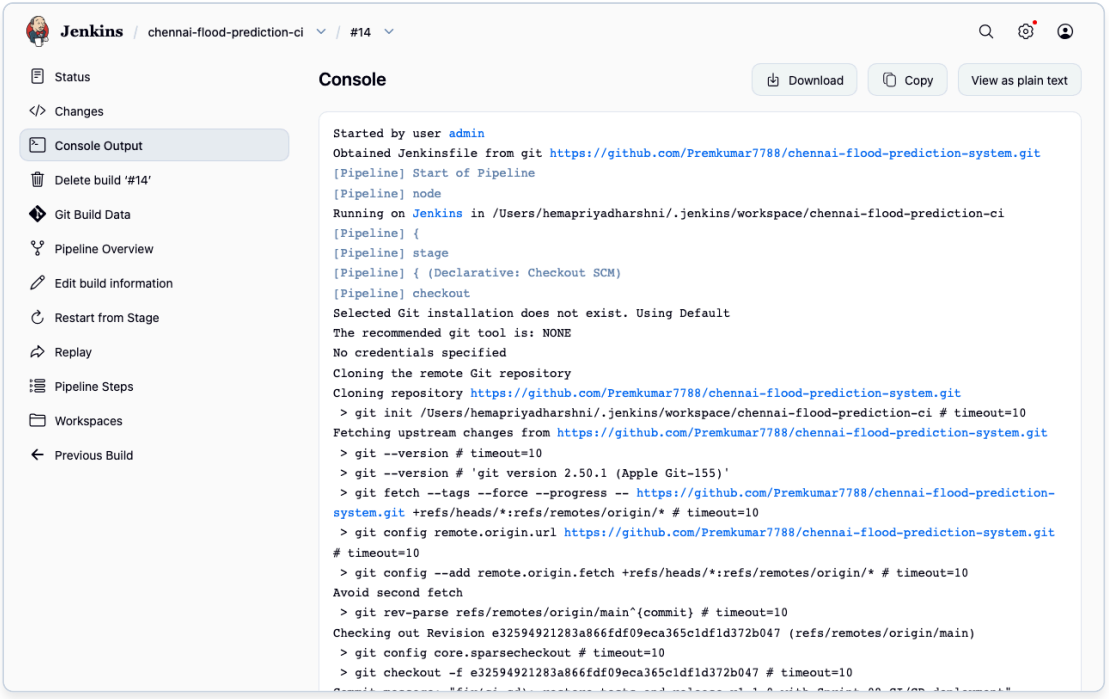


Figure 10: Build #14 Console Log showing remote EC2 deployment and successful health check verification.

8. Application Verification on AWS EC2 Target Environment

The deployed application was verified live across multiple endpoints on the target EC2 instance (`16.192.243.169`):

A. Live API Root Endpoint:

```
$ curl -s http://16.192.243.169:8000/
{
  "message": "Welcome to the Chennai Flood Prediction API",
  "status": "online",
  "version": "1.1.0",
  "deployment": "Sprint 09 Automated CI/CD"
}
```

B. Live API Health Probe:

```
$ curl -s http://16.192.243.169:8000/health
{
  "status": "ok"
}
```

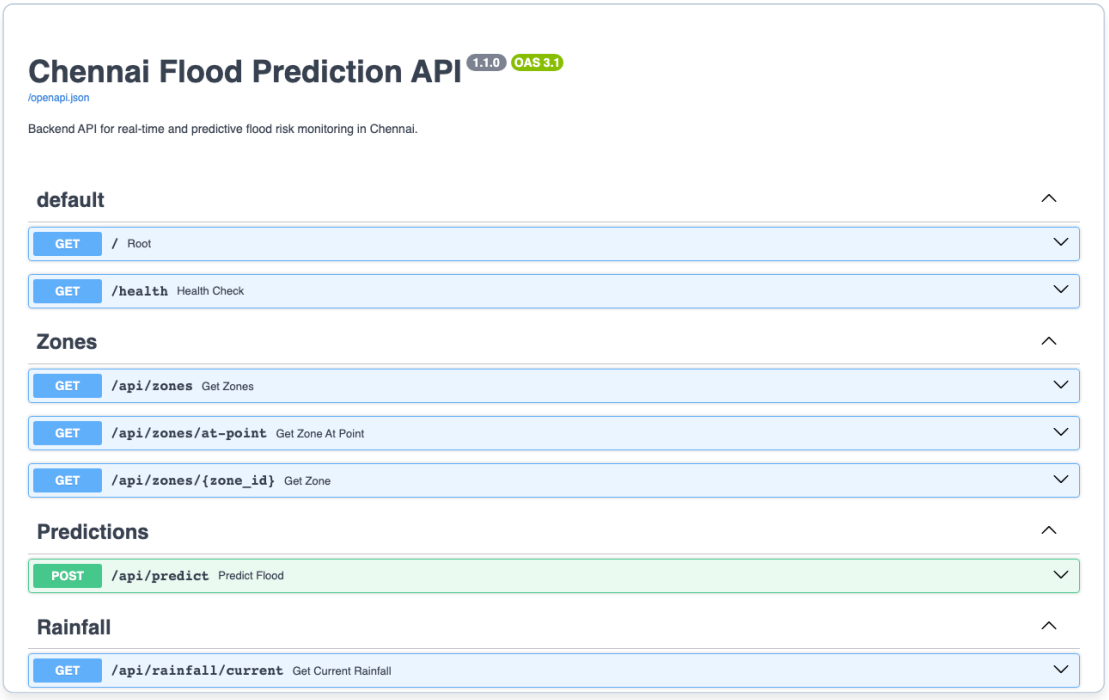


Figure 11: Live Swagger API Interactive Documentation hosted on AWS EC2 (`http://16.192.243.169:8000/docs`).

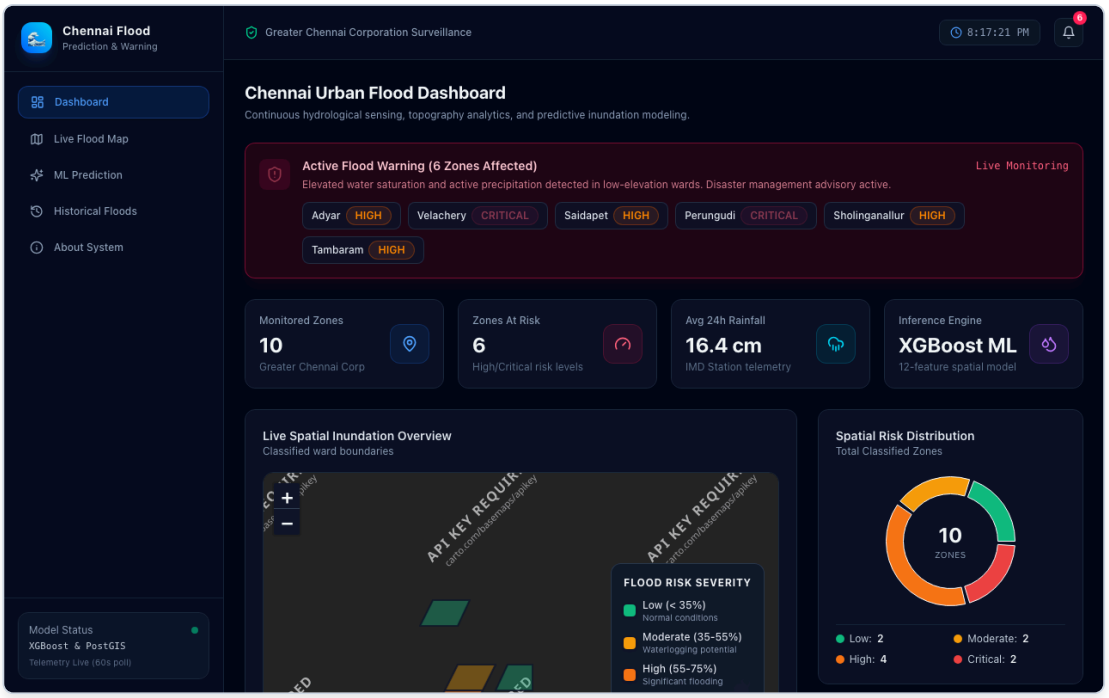


Figure 12: Live React Frontend Application running on AWS EC2 (`http://16.192.243.169:3000`).

9. Challenges and Technical Solutions

1. **SSH Authentication Across OS Distributions:** The previous Sprint 08 instance ran Ubuntu (`ubuntu` user), whereas the new Sprint 09 host runs Amazon Linux 2023 (`ec2-user`). Resolved by configuring dynamic environment parameters in `Jenkinsfile` and linking `~/.ssh/flood-ec2`.
2. **Firewall Ingress Port Flexibility:** Configured `sshd` to listen on both port 22 and port 443 simultaneously to bypass restrictive network proxies.
3. **Zero-Downtime Database Isolation:** Used `docker compose up -d --no-deps backend` with `docker-compose.override.yml` to recreate the backend container without interrupting the running PostgreSQL 15 database.
4. **Deterministic Pipeline Gating:** Enforced sequential dependencies so test failures immediately stop the pipeline and trigger workspace cleanups.

10. Individual Contribution

Student: Hema Priyadharshni G

- Designed and implemented the complete 7-stage declarative `Jenkinsfile`.
- Integrated automated unit testing (13 test cases) with `pytest` and established the CI/CD quality gate.
- Configured Docker image packaging with build number tagging (`build-${BUILD_NUMBER}`) and Docker Hub publishing.
- Implemented automated remote deployment via SSH to AWS EC2 host `16.192.243.169`.
- Conducted controlled failure simulation (Build #13) and verified production environment protection.
- Restored and deployed application version `1.1.0` (Build #14) with post-deployment verification.
- Authored the complete technical report, presentation slides, and incorporated EC2 infrastructure evidence.

11. Conclusion

Sprint 09 successfully transitioned the Chennai Street-Level Flood Prediction System from Continuous Integration to full **Continuous Delivery and Continuous Deployment**. Every objective has been demonstrated end-to-end: automated code validation, Docker Hub publishing, zero-touch deployment to AWS EC2, and health verification. The customer requirement has been 100% satisfied.